

UNIVERSIDAD/INSTITUCIÓN

Deusto Formación

FACULTAD/DEPARTAMENTO

Informática y tecnología

ASIGNATURA

Máster Full Stack

PRÁCTICA: Desarrollo de una API NodeJS con Express y MongoDB

ESTUDIANTE:

Jorge Borrego

FECHA:

20/02/2025

INTRODUCCIÓN

En esta práctica se implementará una API RESTful utilizando NodeJS con Express y MongoDB, con el objetivo de comprender el desarrollo de operaciones CRUD y afianzar los conceptos de asincronía en JavaScript.

OBJETIVOS

- Desarrollar un proyecto NodeJS con Express para gestionar una entidad de teléfonos móviles.
- Implementar la sintaxis de servidor con Express.
- Gestionar peticiones "GET", "PUT", "POST" y "DELETE" con Express.
- Generar una base de datos y colección en MongoDB.
- Conectar la base de datos con la API usando Mongoose.
- Comprobar el correcto funcionamiento mediante Postman y MongoDB Compass.

DESARROLLO

1. Configuración del Proyecto:

- Crear un directorio vacío denominado `evaluacion` y abrirlo con Visual Studio Code.

```
> EVALUACION  
> ESQUEMA  
> LÍNEA DE TIEMPO
```

- Inicializar un proyecto NodeJS ejecutando:

```
Proyectos\evaluacion> npm init -y
```

- Instalar las dependencias necesarias:

```
npm i nodemon --save-dev
```

```
npm i express --save
```

```
npm i mongoose --save
```

```
npm install dotenv --save
```

```
npm install swagger-jsdoc swagger-ui-express --save
```

```
npm install ymljs
```

- Agregar un script en package.json para utilizar nodemon:

```
{
  "name": "evaluacion",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "Run by the 'npm test' command.",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "NodeJs",
    "Express",
    "Mongoose"
  ],
  "author": "Devormar",
  "license": "ISC",
  "description": "Desarrollo de una API NodeJS con Express y MongoDB",
  "devDependencies": {
    "nodemon": "^3.1.9"
  },
  "dependencies": {
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "mongoose": "^8.10.1",
    "swagger-jsdoc": "^6.2.8",
    "swagger-ui-express": "^5.0.1",
    "yamljs": "^0.3.0"
  }
}
```

- Crear un archivo .gitignore e incluir:

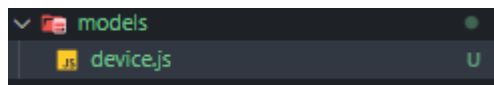
```
.gitignore
node_modules
.env
```

2. Creación del Modelo

- Crear un directorio models en la raíz del proyecto.

```
models
```

- Crea un directorio llamado models en la raíz del proyecto con un archivo device.js que contenga el esquema Mongoose para la entidad.



```
const mongoose = require('mongoose');

const DeviceSchema = new mongoose.Schema(
  {
    name : {
      type : String,
      required: true,
    },
    art_number : {
      type : String,
      required: [true, 'article number is required'],
      trim : true,
      validate: {
        validator: function(v) {
          return /^[A-Z0-9]{12}$/.test(v);
        },
        message: props => `${props.value} is not a valid article number!`
      }
    },
    serial_number : {
      type : String,
      required: [true, 'serial number is required'],
      unique : true,
      trim : true,
      validate: {
        validator: function(v) {
          return /^[A-Z0-9]{14}$/.test(v);
        },
        message: props => `${props.value} is not a valid serial number!`
      }
    },
    description : {
      type : String,
      required: true,
      validate: {
        validator: function(v) {
          return /^.{0,40}$/.test(v);
        },
        message: props => `${props.value} Max. 40 caracteres`
      }
    },
    price : {
      type : Number,
      required: [true, 'Why no price?'],
      min : 0.00,
      max : 99999.99,
      validate: {
        validator: function(v) {
          return /^\d+(\.\d{1,2})?$/.test(v);
        },
        message: props => `${props.value} is not a valid price`
      }
    }
  }
);

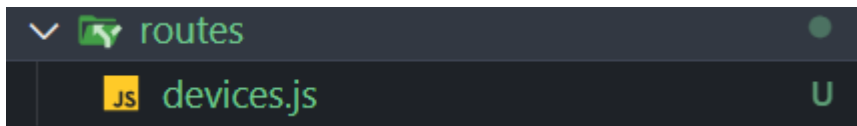
module.exports = mongoose.model('Device', DeviceSchema);
```

3. Creación de las Rutas

- Crear un directorio routes en la raíz del proyecto.



- Crea un directorio llamado routes en la raíz del proyecto con un archivo devices.js que contenga los métodos para peticiones "get", "put", "post" y "delete" con Express enlazados con el esquema de Mongoose usando el campo "_id" para los métodos "put" y "delete".



```
const express = require('express');
const app = express.Router();

const Device = require('../models/device');

app.get('/', async (req, res) => {
  try {
    const devices = await Device.find();
    res.status(200).json({
      message: 'Ok',
      devices
    })
  } catch (error) {
    res.status(500).json({
      message: 'Database Error',
      Error: `${error}`
    })
  }
})

app.post('/', async (req, res) => {
  try {
    const newDevice = new Device(req.body);
    await newDevice.save();
    res.status(200).json({
      message: 'Ok',
      DeviceCreated: newDevice
    })
  } catch (error) {
    res.status(500).json({
      message: 'Database Error',
      Error: `${error}`
    })
  }
})

app.put('/', async (req, res) => {
  try {
    const deviceUpdated = await Device.findOneAndUpdate(
      { _id: req.body._id }, req.body, { upsert: true, new: true, runValidators: true });
    res.status(200).json({
      message: 'Ok',
      deviceUpdated
    })
  } catch (error) {
    res.status(500).json({
      message: 'Database Error',
      Error: `${error}`
    })
  }
})

app.delete('/:_id', async (req, res) => {
  try {
    const deviceDeleted = await Device.findOneAndDelete({ _id: req.params._id }, );
    res.status(200).json({
      message: 'Ok',
      deviceDeleted
    })
  } catch (error) {
    res.status(500).json({
      message: 'Database Error',
      Error: `${error}`
    })
  }
})

module.exports = app;
```

4. Configuración del Servidor

- En la raíz del proyecto, crea el archivo principal app.js con la sintaxis de servidor de Express.

```
JS app.js U

const dotenv = require('dotenv').config();
const mongoose = require('mongoose');
const express = require('express');
const app = express();

const PORT = process.env.DB_PORT;

app.use(express.json());

app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
})
```

- Crea la conexión con el servidor de base de datos.

```
const mongooseURI = `mongodb://localhost:27017/${process.env.DB_NAME}`;

mongoose.connect(mongooseURI)
  .then(() => console.log('Conectado a BBDD'))
  .catch((err) => console.log(`Error BBDD disconnected ${err}`));
```

- Importa en app.js los métodos desarrollados en el archivo devices.js.

```
const devices = require('./routes/devices');

app.use(express.json());

app.use('/devices', devices);
```

5. Pruebas con Postman y MongoDB Compass

- Levantar el servidor con:

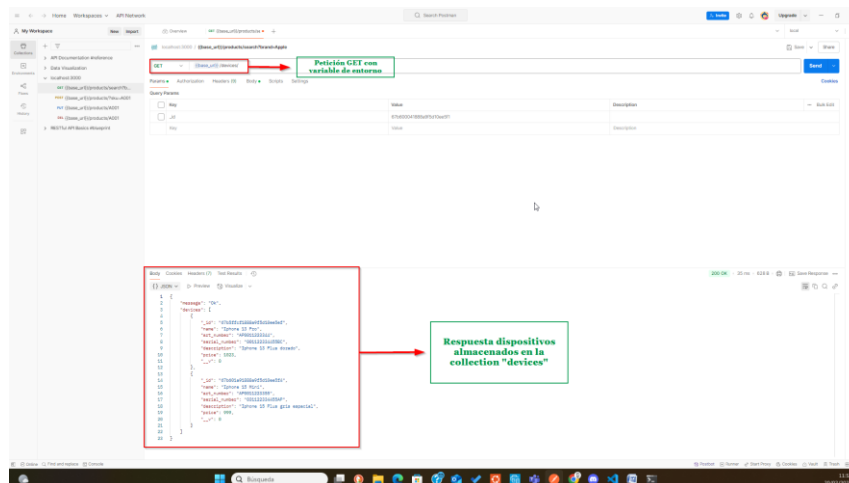
```
PS C:\Users\jorge\Desktop\Master FullStack\Modulos\M7_BasesDeDatos\Proyectos\evaluacion> npm start

[nodemon] 3.1.9
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Servidor escuchando en http://localhost:3000
```

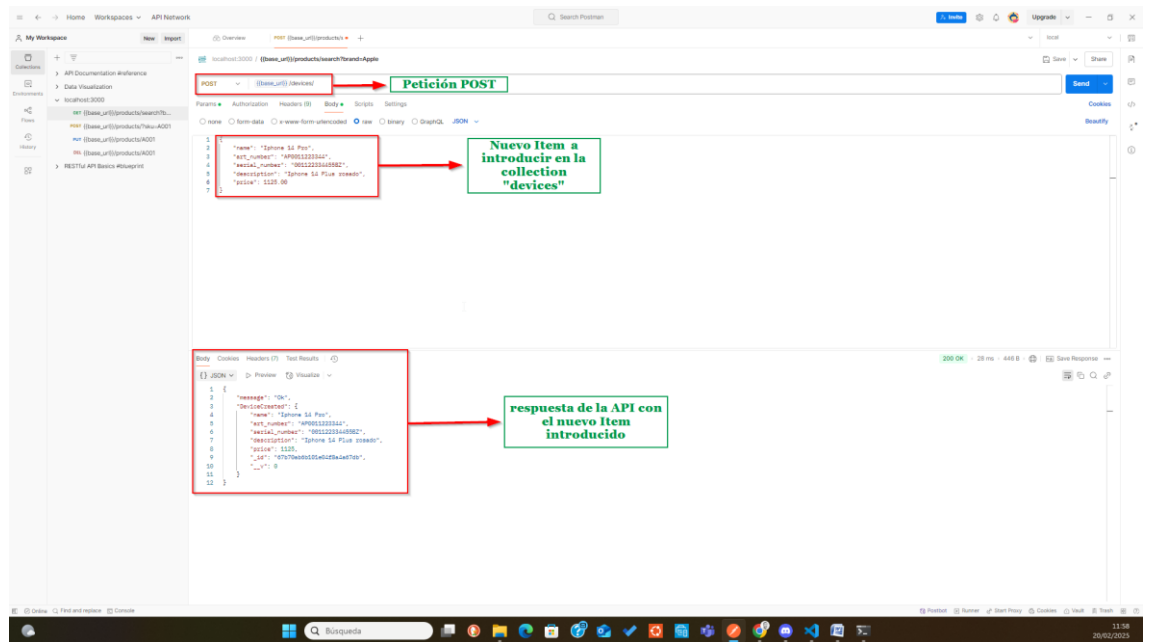
- Levantamos servidor de MongoDB

```
C:\Program Files\MongoDB\>
{"t":{"$date":"2025-02-20T11:48:01.861+01:00"},"s":"I", "c":"NETWORK", "id":23015, "ctx":"listener","msg":"Listening
on","attr":{"address":"127.0.0.1:27017"}}
{"t":{"$date":"2025-02-20T11:48:01.861+01:00"},"s":"I", "c":"NETWORK", "id":23016, "ctx":"listener","msg":"Waiting f
or connections","attr":{"port":27017,"ssl":"off"}}
{"t":{"$date":"2025-02-20T11:48:01.862+01:00"},"s":"I", "c":"CONTROL", "id":8423403, "ctx":"initandlisten","msg":"mong
od startup complete","attr":{"Summary of time elapsed":{"Startup from clean shutdown?":true,"Statistics":{"Set up period
ic runner":"0 ms","Set up online certificate status protocol manager":"0 ms","Transport layer setup":"0 ms","Run initial
syncer crash recovery":"0 ms","Create storage engine lock file in the data directory":"0 ms","Get metadata describing s
torage engine":"0 ms","Validate options in metadata against current startup options":"0 ms","Create storage engine":"466
ms","Write current PID to file":"0 ms","Initialize FCV before rebuilding indexes":"34 ms","Drop abandoned idents and ge
t back indexes that need to be rebuilt or builds that need to be restarted":"0 ms","Rebuild indexes for collections":"0
ms","Load cluster parameters from disk for a standalone":"1 ms","Build user and roles graph":"0 ms","Set up the backgrou
nd thread pool responsible for waiting for opTimes to be majority committed":"0 ms","Start up the replication coordinato
r":"3 ms","Ensure the change stream collections on startup contain consistent data":"0 ms","Write startup options to the
audit log":"0 ms","Start transport layer":"1 ms","_initAndListen total elapsed time":"1514 ms"}}}}
{"t":{"$date":"2025-02-20T11:48:02.008+01:00"},"s":"W", "c":"CONTROL", "id":636300, "ctx":"ftdc","msg":"Use of deprec
ated server parameter name","attr":{"deprecatedName":"internalQueryCacheSize","canonicalName":"internalQueryCacheMaxEntr
iesPerCollection"}}
{"t":{"$date":"2025-02-20T11:48:02.009+01:00"},"s":"W", "c":"CONTROL", "id":636300, "ctx":"ftdc","msg":"Use of deprec
ated server parameter name","attr":{"deprecatedName":"oplogSamplingLogIntervalSeconds","canonicalName":"collectionSampli
ngLogIntervalSeconds"}}
{"t":{"$date":"2025-02-20T11:48:02.009+01:00"},"s":"W", "c":"NETWORK", "id":23803, "ctx":"ftdc","msg":"Use of deprec
ated server parameter 'sslMode', please use 'tlsMode' instead."}
{"t":{"$date":"2025-02-20T11:48:02.010+01:00"},"s":"W", "c":"CONTROL", "id":636300, "ctx":"ftdc","msg":"Use of deprec
ated server parameter name","attr":{"deprecatedName":"wiredTigerConcurrentReadTransactions","canonicalName":"storageEngi
neConcurrentReadTransactions"}}
{"t":{"$date":"2025-02-20T11:48:02.010+01:00"},"s":"W", "c":"CONTROL", "id":636300, "ctx":"ftdc","msg":"Use of deprec
ated server parameter name","attr":{"deprecatedName":"wiredTigerConcurrentWriteTransactions","canonicalName":"storageEngi
neConcurrentWriteTransactions"}}
```

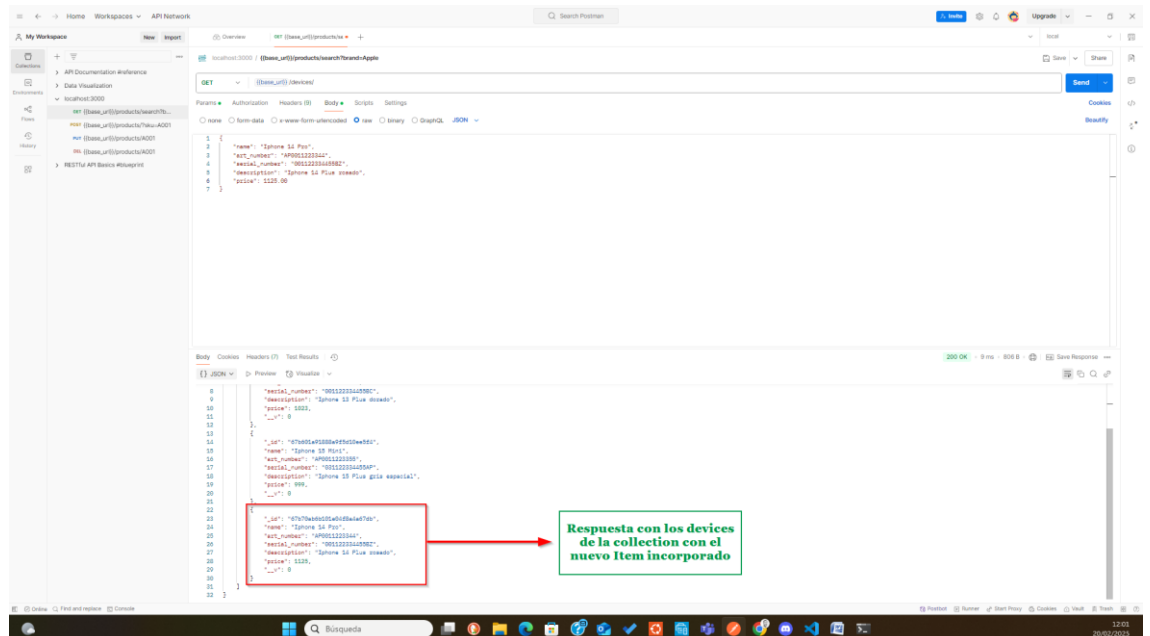
- Usar Postman para probar las siguientes peticiones:
 - GET** <http://localhost:3000/devices> → Obtener todos los dispositivos.



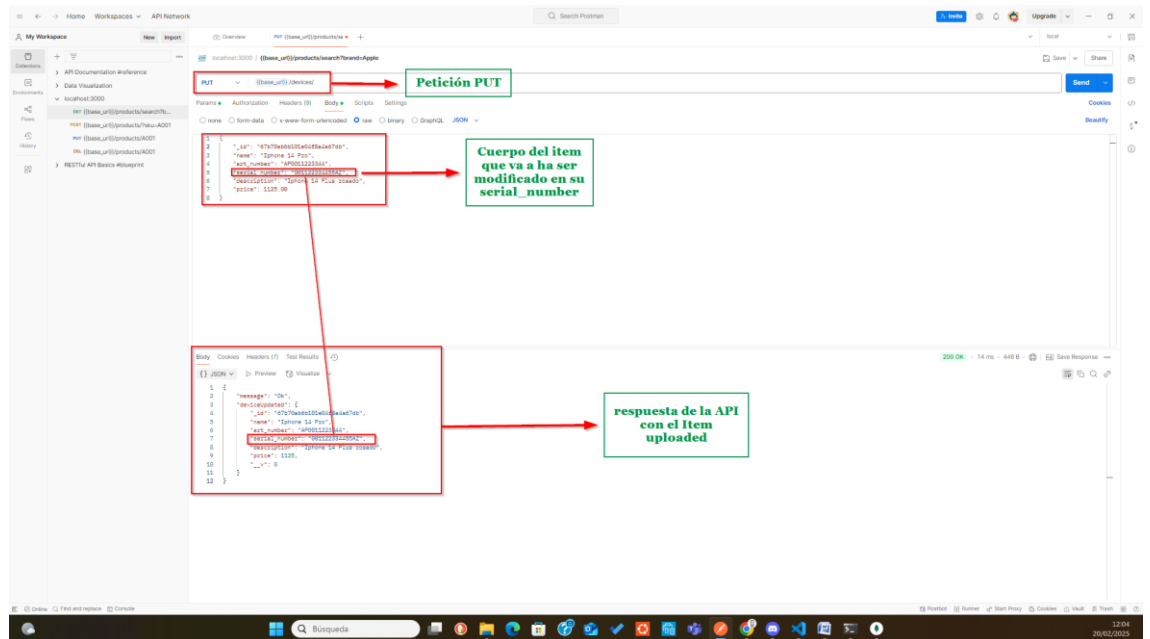
- **POST** <http://localhost:3000/devices> → Agregar un nuevo dispositivo.



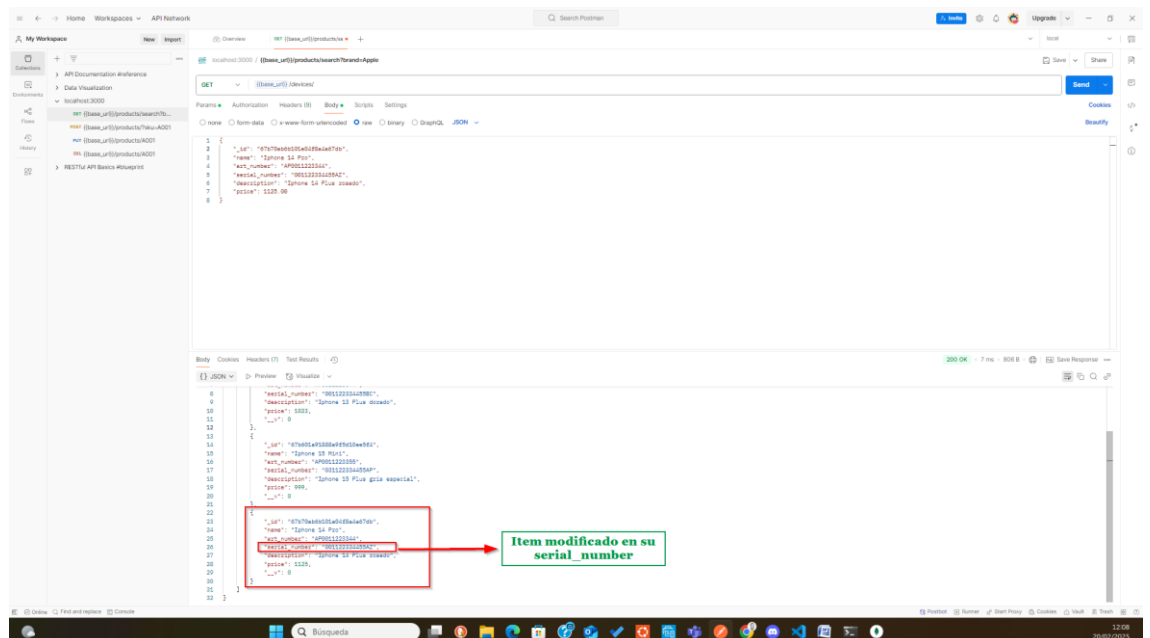
Realizamos un GET para confirmar el POST:



- **PUT** <http://localhost:3000/devices/:id> → Actualizar un dispositivo por ID.



Petición GET para comprobar la petición PUT:



- **DELETE** `http://localhost:3000/devices/:id` → Eliminar un dispositivo por ID

The screenshot shows the Postman interface with a DELETE request configured. The URL is `http://localhost:3000/devices/67b60004188a9f5c0d5e0f`. The method is set to DELETE. A red box highlights the DELETE method and the URL. A green box with an arrow points to the DELETE method with the text "Petición DELETE idenitificador de item '_id'".

The response is shown in the Body tab, displaying a JSON object:


```
{
  "message": "OK",
  "deviceDeleted": {
    "_id": "67b60004188a9f5c0d5e0f",
    "name": "iPhone 13 Pro",
    "art_number": "MP0000000001",
    "serial_number": "000000000000",
    "description": "iPhone 13 Plus verde",
    "price": 999,
    "_v": 0
  }
}
```

 A red box highlights the response body. A green box with an arrow points to the response body with the text "Item borrado de la lista de 'devices'".

Petición Get para comprobar el Item borrado después de la petición Delete:

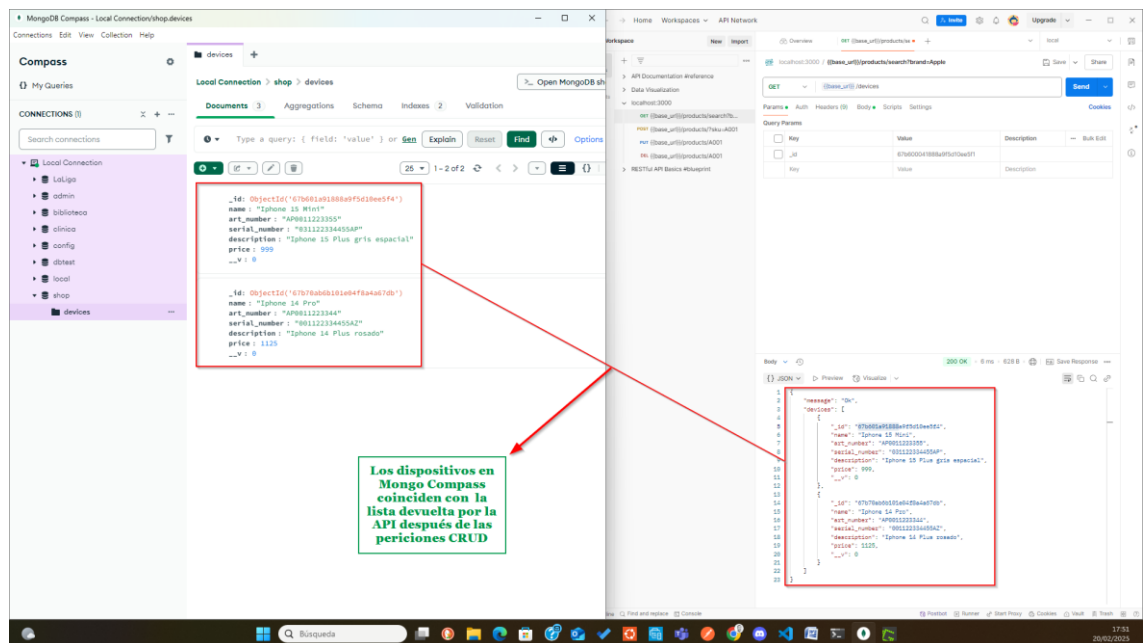
The screenshot shows the Postman interface with a GET request configured. The URL is `http://localhost:3000/devices`. The method is set to GET. A red box highlights the GET method and the URL. A green box with an arrow points to the GET method with the text "Petición Get".

The response is shown in the Body tab, displaying a JSON array:


```
{
  "message": "OK",
  "devices": [
    {
      "_id": "67b60004188a9f5c0d5e0f",
      "name": "iPhone 13 Pro",
      "art_number": "MP0000000001",
      "serial_number": "000000000000",
      "description": "iPhone 13 Plus verde",
      "price": 999,
      "_v": 0
    },
    {
      "_id": "67b60004188a9f5c0d5e0f",
      "name": "iPhone 13 Pro",
      "art_number": "MP0000000001",
      "serial_number": "000000000000",
      "description": "iPhone 13 Plus verde",
      "price": 999,
      "_v": 0
    }
  ]
}
```

 A red box highlights the response body. A green box with an arrow points to the response body with the text "El Item no aparece en la lista de 'devices'".

- **Coincidencia de los dispositivos con Mongo Compass:**



CONCLUSIONES

- Se logró desarrollar una API RESTful con NodeJS, Express y MongoDB.
- Se implementaron correctamente las operaciones CRUD.
- Se reforzaron los conceptos de asincronía y uso de Promesas en JavaScript.
- Se verificó el correcto funcionamiento de la API con Postman y MongoDB Compass.

REFERENCIAS

- Documentación oficial de [Node.js](#)
- Documentación de [Express](#)
- Documentación de [Mongoose](#)
- Guía de uso de [Postman](#)